

Unit I

Introduction to Malware and Malware Analysis : Static Analysis

Dr. Manu V.T.

Assistant Professor (Cyber Security & Digital Forensics), NFSU Bhopal

Static Analysis of Malware

- Overview of static analysis techniques used in malware analysis

Static Analysis

- Static analysis involves examining malware without executing it.
- Used to understand structure, behavior, and intent safely.

Hashing

- Hashing generates unique identifiers for files.
- Common algorithms: MD5, SHA-1, SHA-256.
- Used for malware identification and comparison.

Finding Strings

- Strings are readable text embedded in binaries.
- Can reveal URLs, file paths, commands, or IP addresses.

Obfuscated Strings

- Malware often obfuscates strings to evade detection.
- Decoded to understand malicious intent.

Why Malware Authors Obfuscate Strings

- **Evasion:**
 - ~ Prevent antivirus engines and YARA rules from matching known indicators (URLs, commands, mutex names).
- **Anti-analysis:**
 - ~ Slow down analysts by hiding meaningful context.
- **Operational secrecy:**
 - ~ Conceal C2 servers, file paths, registry keys, or API usage.

Common String Obfuscation Techniques

- Encoding:
 - ~ Base64, hexadecimal, custom alphabets.
- Encryption:
 - ~ AES, RC4.
- String splitting:
 - ~ Breaking strings into fragments and reconstructing them at runtime.
- Dynamic generation:
 - ~ Constructing strings character-by-character during execution.
- Compression:
 - ~ Packing strings and decompressing them in memory.

Indicators of Obfuscated Strings During Static Analysis

- Very few readable strings in a suspicious binary.
- Presence of repetitive arithmetic or XOR operations.
- Loops manipulating byte arrays before API calls.
- Suspicious functions that output strings just before network or file operations.

Static Analysis Approach

- Initial string extraction:
 - ~ Use basic string utilities to identify any plaintext remnants.
- Entropy review:
 - ~ High-entropy data sections often indicate encrypted or encoded strings.
- Code inspection:
 - ~ Identify routines that transform byte arrays into readable buffers.
- Automated decoding:
 - ~ Use specialized tools to recover obfuscated strings without execution.

FLOSS Tool

- FLOSS (FireEye Labs Obfuscated String Solver).
- Used to automatically decode obfuscated strings.
-

“FLOSS”

Algorithm used by FLOSS

- 1) Analyze control flow of malware to identify functions, basic blocks, etc.
- 2) Use heuristics to find potential decoding routines
- 3) Brute force emulate all code paths among basic blocks and functions
- 4) Snapshot emulator state (registers, memory) at appropriate points
- 5) Extract arguments to decoder functions from emulator snapshots
- 6) Emulate decoder functions using extracted arguments and emulator state
- 7) Diff memory state from before and after decoder emulation
- 8) Extract human-readable strings from memory state difference

PE Files

- Portable Executable (PE) format used in Windows.
- Includes headers and sections describing file structure.
- Based on the COFF file format (Common Object File Format).
- Not only .exe files are PE files, dynamic link libraries (.dll), Kernel modules (.srv), Control panel applications (.cpl) and many others are also PE files.

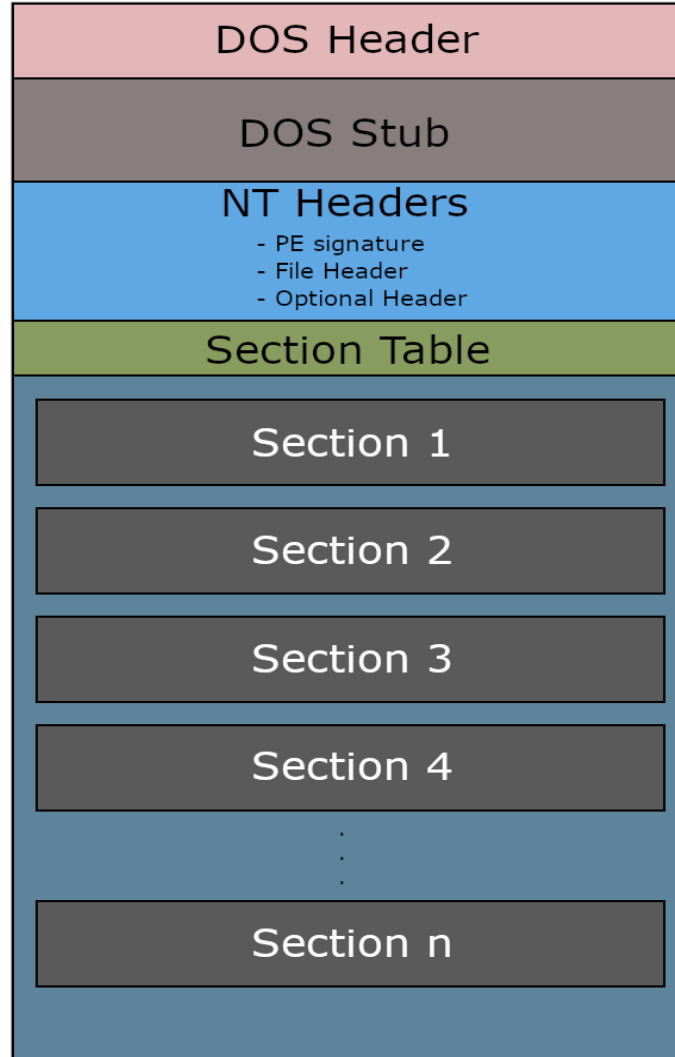
PE Files

- PE (Portable Executable) file format is a data structure that tells the Windows OS loader what information is required to manage the wrapped executable code.
- The data structures on disk are the same data structures used in the memory, and if you know how to find something in a PE file, you can almost certainly find the exact information after the file is loaded into the memory.

Importance in Malware Analysis

- Most Windows malware uses PE format
- Enables static inspection without execution
- Helps identify anomalies and malicious intent

PE Files

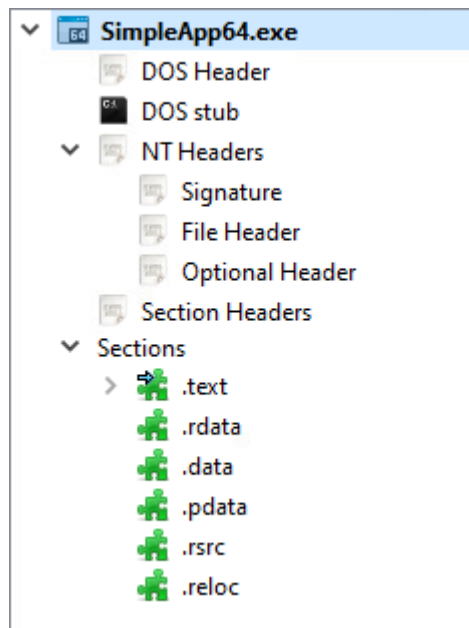


PE Files

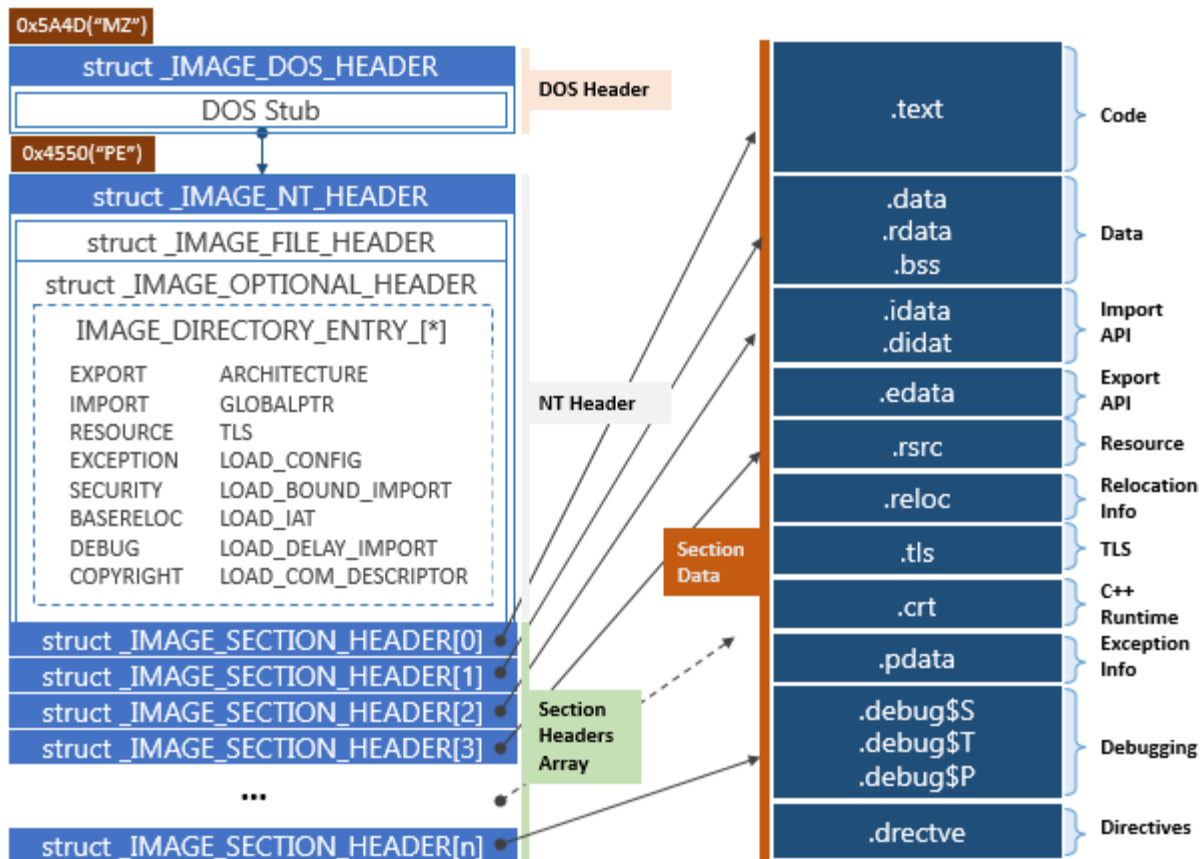
- PE files are not just mapped into memory as a single memory-mapped file.
- Win32 loader looks at the PE file and decides what portions of the file to map in.
- A module in memory represents all the code, data, and resources from an executable file needed by a process.
- Other parts of a PE file may be read but not mapped in (for instance, relocations).
- Some parts may not be mapped in at all, for example, when debug information is placed at the end of the file.
- A field in the PE header tells the system how much memory needs to be set aside for mapping the executable into memory.
- Data that won't be mapped in is placed at the end of the file, past any parts that will be mapped in.

PE Files

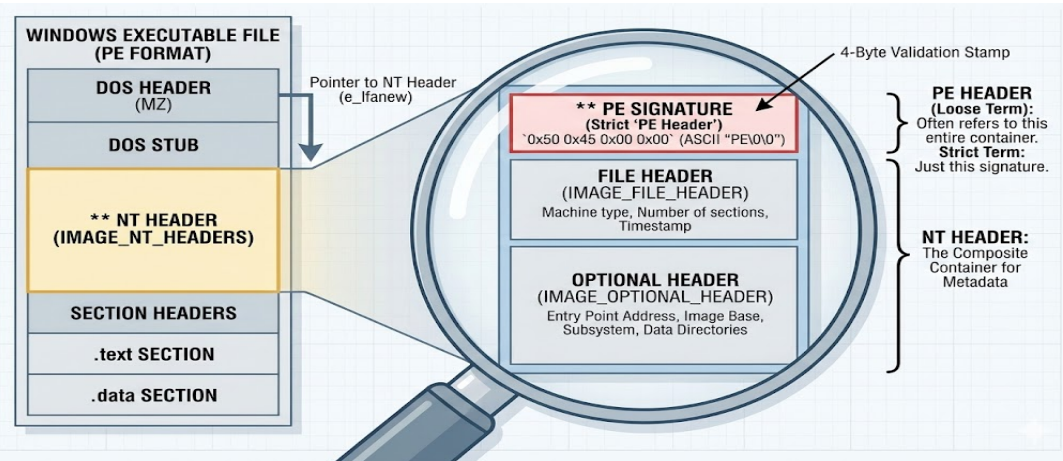
- If we open an executable file with PE-bear
-



PE Format



NT Header & PE Header



- The NT Header is a large data structure containing almost all the vital information the Windows OS needs to load the program.
- The PE Header (strictly speaking) is just a tiny, 4-byte signature located at the very beginning of the NT Header that confirms the file is actually a PE file.

PE Files

- DOS Header

~ Every PE file starts with a 64-bytes-long structure called the DOS header, it's what makes the PE file an MS-DOS executable.

- DOS Stub

~ After the DOS header comes the DOS stub which is a small MS-DOS 2.0 compatible executable that just prints an error message saying "This program cannot be run in DOS mode" when the program is run in DOS mode.

PE Files

- NT Headers

- ~ The NT Headers part contains three main parts:

- PE signature:

- ~ A 4-byte signature that identifies the file as a PE file.

- File Header:

- ~ A standard COFF File Header. It holds some information about the PE file.

- Optional Header:

- ~ The most important header of the NT Headers, its name is the Optional Header because some files like object files don't have it, however it's required for image files (files like .exe files).

- ~ This header provides important information to the OS loader.

PE Files

Section Table

- The section table follows the Optional Header immediately, it is an array of Image Section Headers, there's a section header for every section in the PE file.
- Each header contains information about the section it refers to.

Sections

- Sections are where the actual contents of the file are stored
- These include things like **data** and **resources** that the program uses
- Actual code of the program
- There are several sections each one with its own purpose.

Common PE Sections

- .text – executable code
- .data – global variables
- .rdata – read-only data
- .rsrc – resources

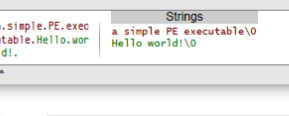
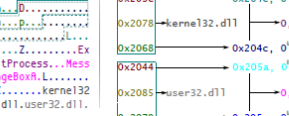
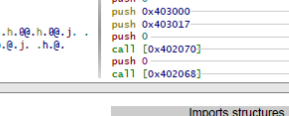
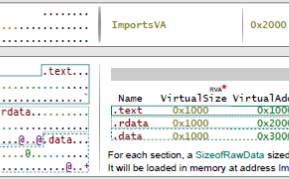
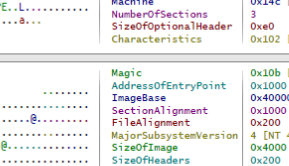
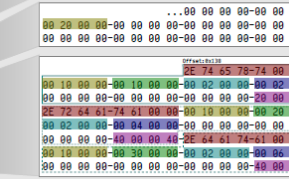
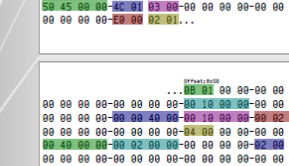
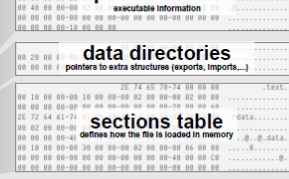
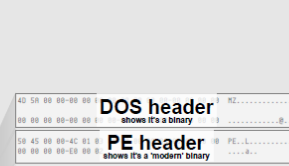
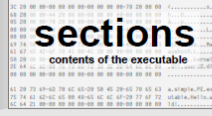
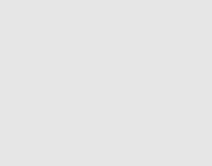
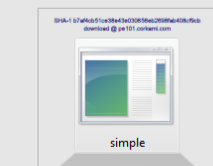
PE in Disk vs Memory

- Disk layout differs from memory layout
- **Loader** maps sections into memory
- Permissions applied at **runtime**

PE Headers and Sections

- Headers contain metadata and execution info.
- Sections include .text, .data, .rdata, .rsrc, etc.

Dissected PE



This is the whole file, however, most PE files contain more elements. Explanations are simplified, for conciseness.

version 1, 3rd May 2012

Loading process

1 Headers

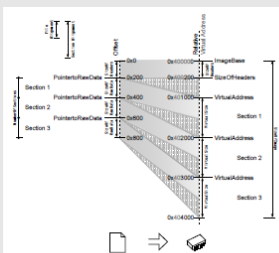
The DOS Header is parsed
the PE Header is parsed
(its offset is DOS Header's e_lfanew)
the Optional Header is parsed
(it follows the PE Header)

2 Sections table

Sections table is parsed
(it is located at: offset (OptionalHeader) + SizeOfOptionalHeader)
it contains NumberOFSections elements
it is checked for validity with alignments:
FileAlignments and SectionAlignments

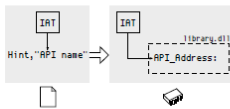
3 Mapping

The file is mapped in memory according to:
the ImageBase
the SizeOfHeaders
the Sections table



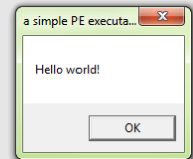
4 Imports

DataDirectories are parsed
they follow the OptionalHeader
their number is NumOFRVAAndSizes
imports are always #2
Imports are parsed
each descriptor specifies a DLLname
this DLL is loaded in memory
IAT and INT are parsed simultaneously
for each API in INT
its address is written in the IAT entry



5 Execution

Code is called at the EntryPoint
the calls of the code go via the IAT to the APIs



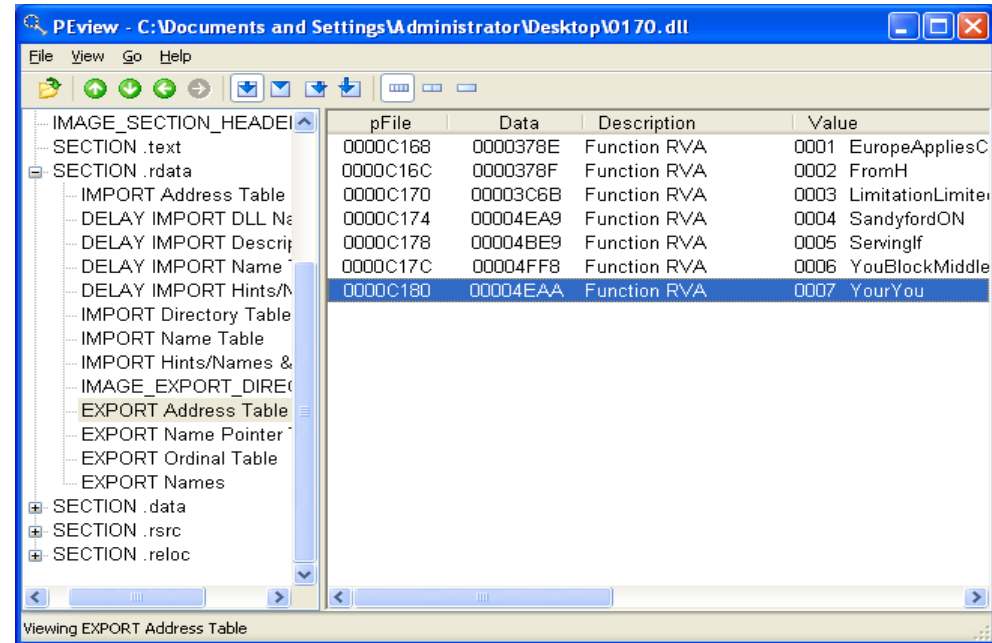
Notes

MZ HEADER aka DOS_HEADER
Starts with 'MZ' (initials of Mark Zibowski MS-DOS developer)
PE HEADER aka IMAGE_FILE_HEADERS / COFF file header
Starts with 'PE' (Portable Executable)
OPTIONAL HEADER aka IMAGE_OPTIONAL_HEADER
Optional only for non-standard PE's but required for executables
RVA Relative Virtual Address
Address relative to ImageBase (at ImageBase, RVA = 0)
Almost all addresses of the headers are RVAs
In code, addresses are not relative.

INT Import Name Table
Null-terminated list of pointers to Hint, Name structures
IAT Import Address Table
Null-terminated list of pointers
On file it is a copy of the INT
After loading it points to the imported APIs
HINT
Index in the exports table of a DLL to be imported
Not required but provides a speed-up by reducing look-up

PE View

- Used to inspect PE file structure visually.
- It is developed and actively maintained by Wayne J. Radburn, who also has some other neat software you can find on his website
- Helps analyze imports, exports, and sections.



Linked Libraries and Functions

- Imported DLLs and APIs indicate malware capabilities.
- Examples: networking, file access, registry operations.

Identification of Malware Capabilities

DLL	Indicates
kernel32.dll	Process, memory, and file operations
advapi32.dll	Registry access, privilege escalation
user32.dll	Keylogging, screen interaction
ws2_32.dll	Network and C2 communication
wininet.dll	HTTP/HTTPS-based data exfiltration
ntdll.dll	Low-level system calls, evasion techniques

Dependency Walker

- Tool to analyze imported and exported functions.
- Helps identify suspicious API usage.
- Download Link:
<https://www.dependencywalker.com/>

Dependency Walker

- Free utility that scans any 32-bit or 64-bit Windows module (exe, dll, ocx, sys, etc.) and builds a hierarchical tree diagram of all dependent modules.
- For each module found:
 - ~ lists all the functions that are exported by that module
 - ~ which of those functions are actually being called by other modules.
- Another view displays the minimum set of required files, along with detailed information about each file including a
 - ~ full path to the file
 - ~ base address
 - ~ version numbers
 - ~ machine type
 - ~ debug information, and more.

Dependency Walker

- Useful for troubleshooting system errors related to loading and executing modules.
- Detects many common application problems such as
 - ~ missing modules
 - ~ invalid modules
 - ~ import/export mismatches
 - ~ circular dependency errors
 - ~ mismatched machine types of modules, and
 - ~ module initialization failures

Dependency Walker

The screenshot shows the Dependency Walker application window for the file **STOOGES.EXE**. The interface includes a menu bar (File, Edit, View, Options, Profile, Window, Help), a toolbar, and a tree view on the left showing the dependency hierarchy. The tree shows **STOOGES.EXE** depending on **LARRY.DLL**, **KERNEL32.DLL**, **NTDLL.DLL**, **CURLY.DLL**, **SHEMP.DLL**, and **MOE.DLL**. **CURLY.DLL** is currently selected.

On the right, there are two tables showing function information:

PI^	Ordinal	Hint	Function	Entry Point
	N/A	N/A	IsKnucklehead	Not Bound
	N/A	N/A	int SaySoitenly(char *,...)	Not Bound

E^	Ordinal	Hint	Function	Entry Point
	4 (0x0004)	1 (0x0001)	int SaySoitenly(char *,...)	SHEMP.?SaySoitenly@@YAHP/
	5 (0x0005)	2 (0x0002)	DoinkLarrysEye	0x00001010
	3 (0x0003)	0 (0x0000)	void SayPoifect(_int64)	0x00001020
	1 (0x0001)	N/A	N/A	0x00001020
	2 (0x0002)	3 (0x0003)	DoinkMoesEye	SHEMP.DoinkMoesEye

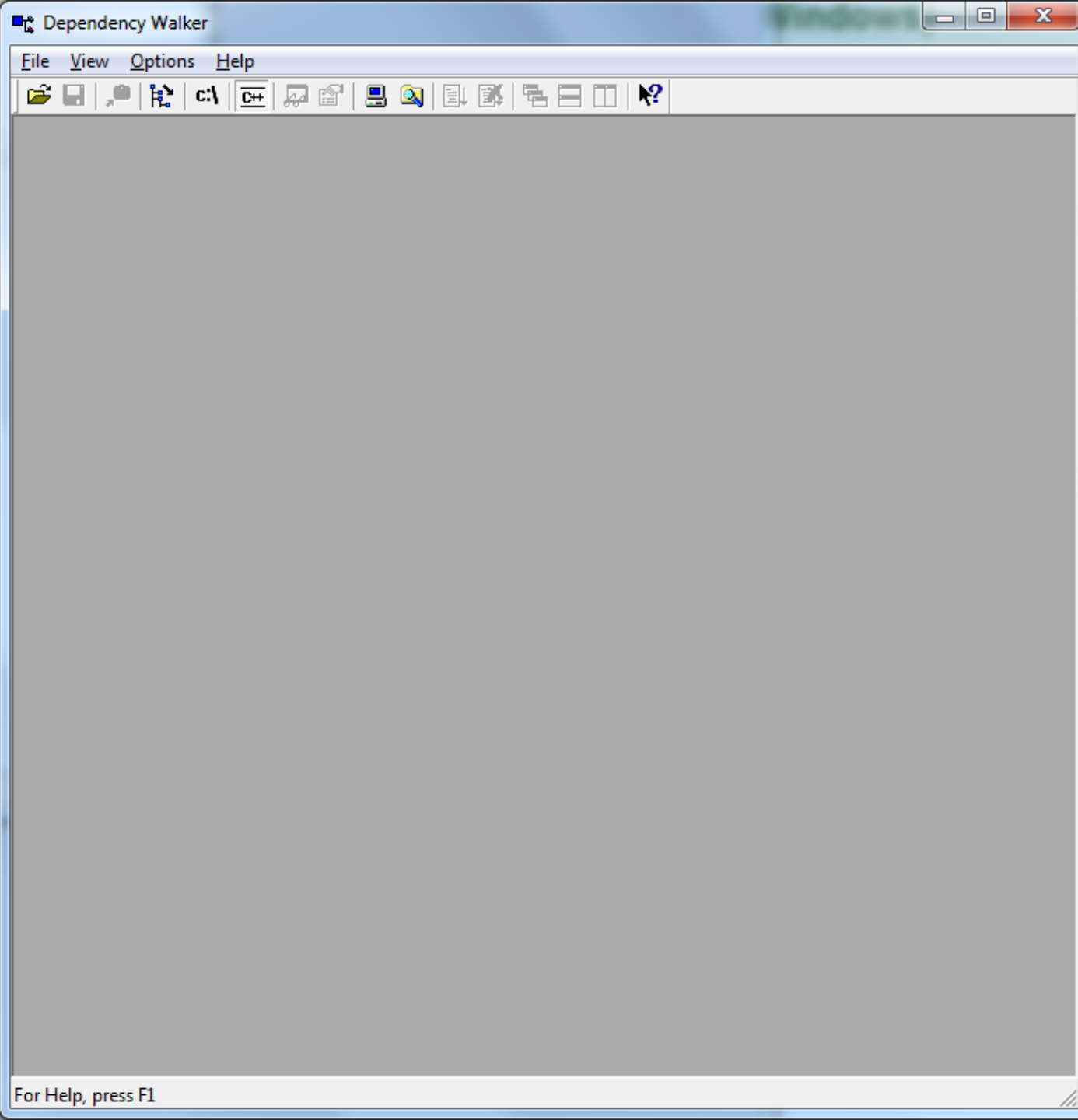
	Module ^	File Time Stamp	Link Time Stamp	File Size	Attr.	Link Checksum	Real Checksum	CPU	Subsystem ^
	CURLY.DLL	11/14/2006 5:17p	11/14/2006 5:13p	2,560	A	0x0000F739	0x0000F759	x86	GUI
	KERNEL32.DLL	08/30/2006 1:22a	08/30/2006 1:20a	871,424	A	0x000E388E	0x000E388E	x86	Console
	LARRY.DLL	11/14/2006 5:13p	11/14/2006 5:13p	2,560	A	0x000053DB	0x000053DB	x86	GUI
	MOE.DLL	11/14/2006 5:15p	11/14/2006 5:15p	2,560	A	0x0000B191	0x0000B191	x86	GUI
	NTDLL.DLL	08/30/2006 1:23a	08/30/2006 1:21a	1,147,664	A	0x00125FA5	0x00125FA5	x86	Console
	SHEMP.DLL	11/14/2006 5:13p	11/14/2006 5:13p	2,560	A	0x00001CE7	0x00001CE7	x86	GUI

Log of events:

```

00:00:00.093: LoadLibraryA("Moe.dll") called from "STOOGES.EXE" at address 0x00401024 by thread 1.
00:00:00.093: Loaded "MOE.DLL" at address 0x00020000 by thread 1. Successfully hooked module.
00:00:00.093:DllMain(0x00020000, DLL_PROCESS_ATTACH, 0x00000000) in "MOE.DLL" called by thread 1.
00:00:00.093:DllMain(0x00020000, DLL_PROCESS_ATTACH, 0x00000000) in "MOE.DLL" returned 1 (0x1) by thread 1.
00:00:00.093: LoadLibraryA("Moe.dll") returned 0x00020000 by thread 1.
00:00:00.109: GetProcAddress(0x00020000 [MOE.DLL], "SmackCurly") called from "STOOGES.EXE" at address 0x0040102B and returne
  
```

For Help, press F1



Dependency Walker - [addressbook.exe]

File Edit View Options Profile Window Help

ADDRESSBOOK.EXE

- QTCORE4.DLL
 - KERNEL32.DLL
 - USER32.DLL
 - OLE32.DLL
 - ADVAPI32.DLL
 - MSVCRT.DLL
 - NTDLL.DLL
 - KERNELBASE.DLL
 - API-MS-WIN-SERVICE-CORE-L1-1-0.DLL
 - API-MS-WIN-SERVICE-WINSVC-L1-1-0.DLL
 - API-MS-WIN-SERVICE-MANAGEMENT-L1-1-0.DLL
 - API-MS-WIN-SERVICE-MANAGEMENT-L1-1-0.DLL
 - API-MS-WIN-CORE-LOCALREGISTRY-L1-1-0.DLL
 - API-MS-WIN-CORE-NAMEDPIPE-L1-1-0.DLL
 - API-MS-WIN-CORE-PROCESSTHREADS-L1-1-0.DLL
 - API-MS-WIN-SECURITY-BASE-L1-1-0.DLL

PI	Ordinal	Hint	Function ^	Entry Point
----	---------	------	------------	-------------

E	Ordinal ^	Hint	Function	Entry Point
---	-----------	------	----------	-------------

Module ^	File Time Stamp	Link Time Stamp	File Size	Attr.
ACLUI.DLL	14.07.2009 03:14	14.07.2009 03:03	125.440	A
ACTIVEDS.DLL	20.11.2010 14:18	20.11.2010 13:54	202.752	A
ADDRESSBOOK.EXE	04.11.2010 05:56	04.11.2010 05:56	75.776	A
ADSLDPC.DLL	14.07.2009 03:14	14.07.2009 03:03	202.752	A
ADVAPI32.DLL	20.11.2010 14:18	20.11.2010 13:54	640.512	A
ADVPACK.DLL	14.07.2009 03:14	14.07.2009 03:03	126.464	A
API-MS-WIN-CORE-CONSOLE-L1-1-0.DLL	14.07.2009 03:03	14.07.2009 03:04	3.072	HA

Warning: At least one delay-load dependency module was not found.

Warning: At least one module has an unresolved import due to a missing export function in a delay-load dependent module.

For Help, press F1

Dependency Walker - [addressbook.exe]

File Edit View Options Profile Window Help

ADDRESSBOOK.EXE

- QTCore4.DLL
- QTGUI4.DLL
- MSVCR90.DLL
- KERNEL32.DLL

Highlight Matching Module In List Ctrl+M

Highlight Original Instance Ctrl+K

Highlight Previous Instance Ctrl+B

Highlight Next Instance Ctrl+N

Copy File Name Ctrl+C

Expand All Ctrl+E

Collapse All Ctrl+W

Auto Expand F8

Full Paths F9

View Module in External Viewer Enter

Properties... Alt+Enter

Module ^	Link Time Stamp	File Size	Attr.
ACLUI.DLL	20.11.2010 14:18	202.752	A
ACTIVEDS.DLL	04.11.2010 05:56	75.776	A
ADDRESSBOOK.EXE	14.07.2009 03:14	202.752	A
ADSLDPC.DLL	20.11.2010 14:18	640.512	A
ADVAPI32.DLL	14.07.2009 03:14	126.464	A
ADVPACK.DLL	14.07.2009 03:03	3.072	HA
API-MS-WIN-CORE-CONSOLE-L1-1-0.DLL	14.07.2009 03:03		

Warning: At least one delay-load dependency module was not found.

Warning: At least one module has an unresolved import due to a missing export function in a delay-load dependent module.

Show modules' full path in the module tree view and module list view

Dependency Walker - [addressbook.exe]

File Edit View Options Profile Window Help

Icons: Folder, Save, Print, Run, Build, C:\, C++, DLL, EXE, Assembly, Properties, Find, Help, etc.

Tree View:

- c:\users\clemens\squish_bin\squish-4.0.2-qt46x-win32-msvc9\examples\qt4\addressbook\ADDRESSBOOK.EXE
- c:\users\clemens\squish_bin\squish-4.0.2-qt46x-win32-msvc9\examples\qt4\addressbook\QTCORE4.DLL
- c:\users\clemens\squish_bin\squish-4.0.2-qt46x-win32-msvc9\examples\qt4\addressbook\QTGUI4.DLL
- c:\windows\system32\ACLU.I.DLL
- c:\windows\system32\ACTIVEDS.DLL
- c:\windows\system32\ADSLDPC.DLL
- c:\windows\system32\ADVAPI32.DLL

Export Table:

PI	Ordinal	Hint	Function
368	N/A	(0x0170)	GetCommand
425	N/A	(0x01A9)	GetCurrentProc
426	N/A	(0x01AA)	GetCurrentProc
429	N/A	(0x01AD)	GetCurrentTh
569	N/A	(0x0239)	GetStartupInfo
591	N/A	(0x024F)	GetSystemTime
614	N/A	(0x0266)	GetTickCount
698	N/A	(0x02BA)	InterlockedCo

Import Table:

E	Ordinal	Hint	Function
1	(0x0001)	42 (0x002A)	BaseThreadIni
2	(0x0002)	754 (0x02F2)	InterlockedPu
3	(0x0003)	0 (0x0000)	AcquireSRWL
4	(0x0004)	1 (0x0001)	AcquireSRWL
5	(0x0005)	2 (0x0002)	ActivateActCt
6	(0x0006)	3 (0x0003)	AddAtomA
7	(0x0007)	4 (0x0004)	AddAtomW

Module List:

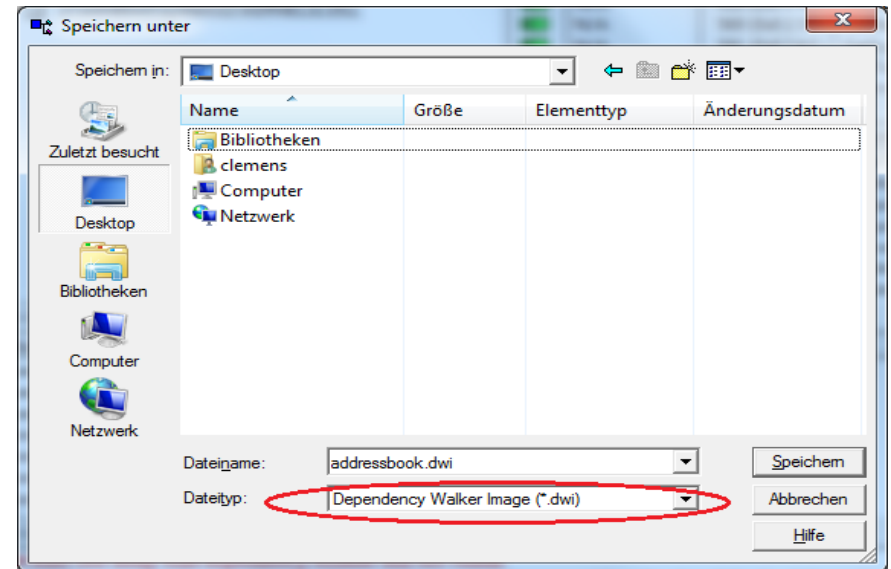
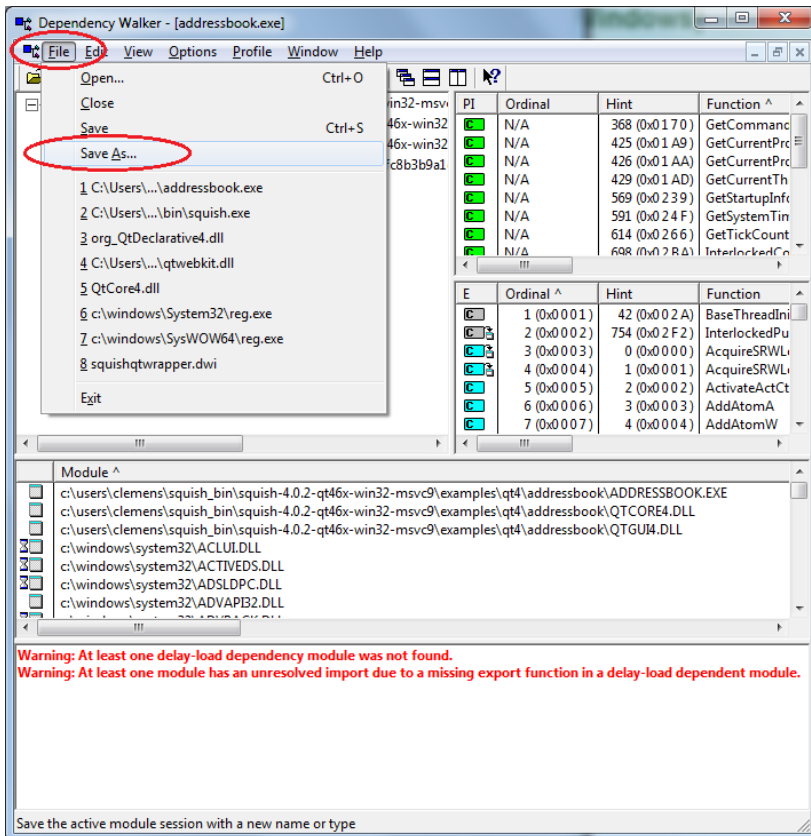
Module ^

- c:\users\clemens\squish_bin\squish-4.0.2-qt46x-win32-msvc9\examples\qt4\addressbook\ADDRESSBOOK.EXE
- c:\users\clemens\squish_bin\squish-4.0.2-qt46x-win32-msvc9\examples\qt4\addressbook\QTCORE4.DLL
- c:\users\clemens\squish_bin\squish-4.0.2-qt46x-win32-msvc9\examples\qt4\addressbook\QTGUI4.DLL
- c:\windows\system32\ACLU.I.DLL
- c:\windows\system32\ACTIVEDS.DLL
- c:\windows\system32\ADSLDPC.DLL
- c:\windows\system32\ADVAPI32.DLL

Warning: At least one delay-load dependency module was not found.

Warning: At least one module has an unresolved import due to a missing export function in a delay-load dependent module.

For Help, press F1



Missing imports/imported symbols

- The information about missing imports is available in the

"Parent Import Function List View"

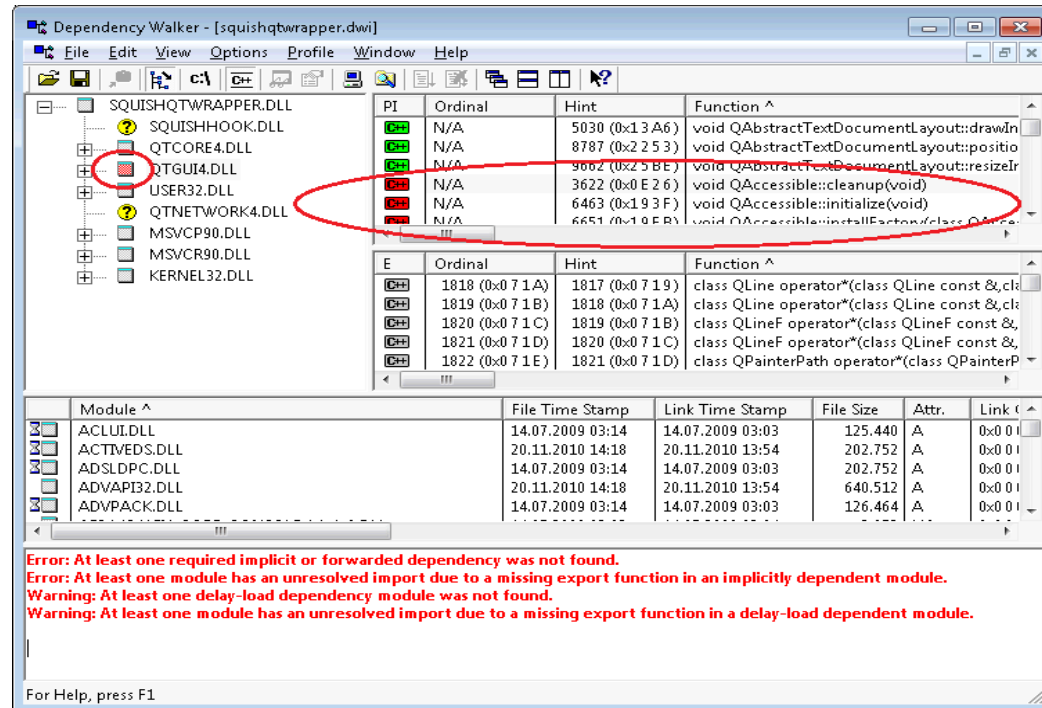
- Since it can be very time consuming to scroll through this possibly very large list to find the missing imports you can use a trick that might make it easier:

Save the information with Save As... in the file format "Text with Import/Export Lists (*.txt)":

Open the file that you just saved in a text editor (since this file can become relatively large be sure to open it with a suitable text editor like Notepad++ or similar).

Search for the name of the .dll with the light red icon (QtGui4.dll in this example).

The second occurrence of this should be the list of imports from this .dll:



[E] QtGui4.DLL

Import	Ordinal	Hint	Function
[+]	N/A	3091 (0x0C13)	void QAbstractButton::animateClick(int)
...			

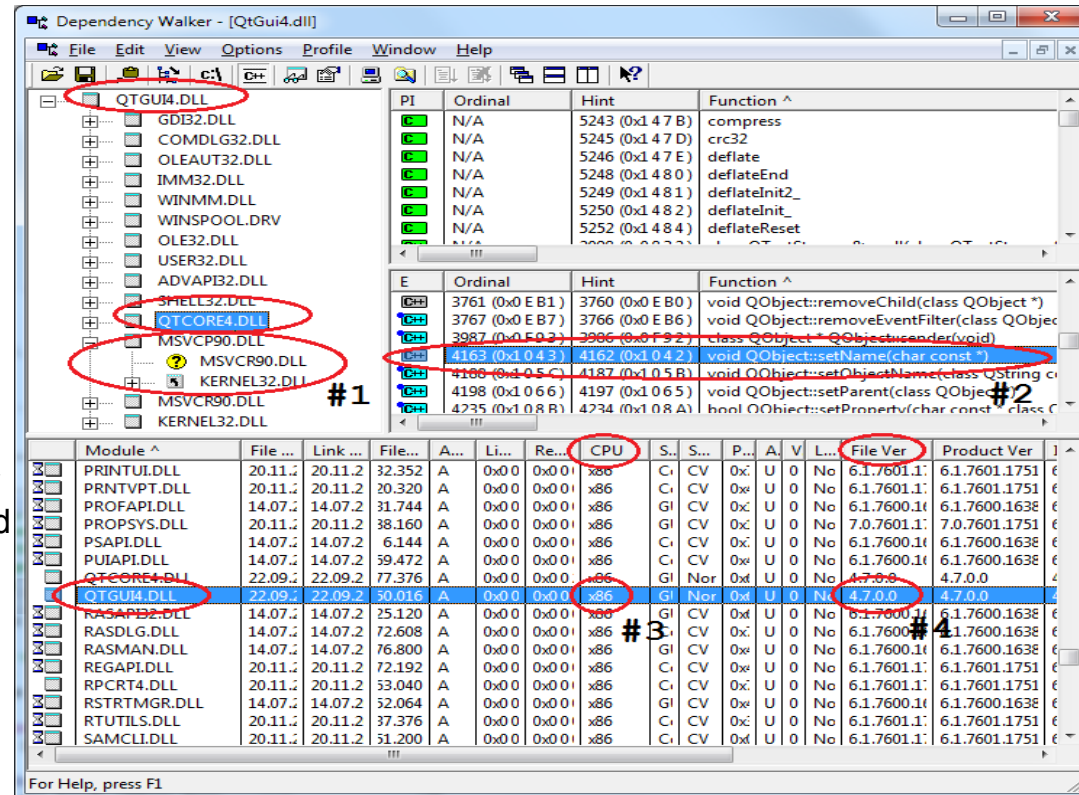
- In this example, QAccessible-related symbols are missing
- In this case this is because the version of Qt being used by the application has been built without the Accessibility module

...
[E] QTGUI4.DLL

Import	Ordinal	Hint	Function
[+]	N/A	3091 (0x0C13)	void QAbstractButton::animateClick(int)
...			
[+]	N/A	9662 (0x25BE)	void QAbstractTextDocumentLayout::resizeInlineObject(class QAbstractTextDocumentLayout::InlineObject*)
[+E]	N/A	3622 (0x0E26)	void QAccessible::cleanup(void)
[+E]	N/A	6463 (0x193F)	void QAccessible::initialize(void)
[+E]	N/A	6651 (0x19FB)	void QAccessible::installFactory(class QAccessibleInterface*)
[+E]	N/A	6652 (0x19FC)	void (*QAccessible::installRootObjectHandler(void (*)(class QAccessible*)))(class QAccessible*)
[+E]	N/A	6654 (0x19FE)	void (*QAccessible::installUpdateHandler(void (*)(class QAccessible*)))(class QAccessible*)
[+E]	N/A	6741 (0x1A55)	bool QAccessible::isActive(void)
...			

Finding information in Dependency Walker

- Runtime libraries of the compiler:
 - ~ Provides compiler version (MSVC6, MSVC7, MSVC8, MSVC9, MSVC10, etc.).
- Build type:
 - ~ Release or Debug build; the latter is indicated by "d" appended to the file names of the Qt DLLs (QT5CORED.DLL, or similar) as well as the Visual C++ Runtime DLLs (MSVCP90D.DLL, or similar).
- Exported symbols:
 - ~ If for example QObject::setName(...) is listed (after selecting QtCore4.dll) the Qt library was configured with enabled Qt3Support module.
- Word size:
 - ~ x86 indicates 32 bit and x64 indicates 64 bit.
- File version:
 - ~ Version of Qt in case of a Qt library.



CFF Explorer

- Advanced PE editor and analyzer.
- Used to inspect headers, sections, and signatures.

CFF Explorer

The screenshot displays the CFF Explorer VII interface, which is used for analyzing Portable Executable (PE) files. The main window shows the file 'CFF Explorer.exe' loaded. The left sidebar contains a tree view of the file's structure, including headers, sections, and various tools like the Address Converter and Dependency Walker. The main pane displays a table of sections, with the '.text' section selected. Below the table, a disassembler window is open, showing the assembly code for the selected section. The disassembler window has a 'Disassembler Parameters' tab, where the disassembler is set to 'x64' and the offset is '1040'. The 'Disassembler Output' pane shows the assembly code for the selected section, including instructions like 'push rsi', 'mov esi, ecx', 'call 0xb6440', 'cmp eax, -0x1', 'jnz 0x13', 'or eax, eax', 'pop rsi', and 'ret 0x4'.

CFF Explorer.exe

Name	Virtual Size	Virtual Address	Raw Size	Raw Address	Reloc Address	Linenumbers	Relocations N...	Linenumbers ...	Characteristics
000001E8	000001F0	000001F4	000001F8	000001FC	00000200	00000204	00000208	0000020A	0000020C
Byte[8]	Dword	Dword	Dword	Dword	Dword	Dword	Word	Word	Dword
.text	001024B5	00001000	00103000	00001000	00000000	00000000	0000	0000	60000020
.rdata	0004B27A	00104000	0004C000	00104000	00000000	00000000	0000	0000	40000040
.data	000106E8	00150000	0000C000	00150000	00000000	00000000	0000	0000	C0000040
.rsrc	000A7E10	00161000	000A8000	0015C000	00000000	00000000	0000	0000	40000040

This section contains:
Code Entry Point: 000D47C6

Section Flags

- ☐ Is shareable
- ☒ Is executable
- ☒ Is readable
- ☐ Is writable
- ☐ Contains extended relocations
- ☐ Can be discarded
- ☐ Is not cachable
- ☐ Is not pageable
- ☐ No pad
- ☒ Contains code
- ☐ Contains initialized data
- ☐ Contains Uninitialized data
- ☐ Contains information
- ☐ Contents won't become part of image
- ☐ Contents comdat

Alignment (Bytes): Default

Quick Disassembler - [CFF Explorer.exe]

Disassembler Parameters

Disassembler: x64

Offset: 1040

Size: DF

Visualization Options

Base Address: 00000000

☒ Show Opcodes

Disassemble

Disassembler Output

Address	Opcode	Instruction
00000000	56	push rsi
00000001	8B F1	mov esi, ecx
00000003	E8 38 64 0B 00	call 0xb6440
00000008	83 F8 FF	cmp eax, -0x1
0000000B	75 06	jnz 0x13
0000000D	0B C0	or eax, eax
0000000F	5E	pop rsi
00000010	C2 04 00	ret 0x4

• CFF Explorer

- CFF Explorer was designed to make PE editing as easy as possible, but without losing sight on the portable executable's internal structure.
- This application includes a series of tools which might help not only reverse engineers but also programmers.
- It offers a multi-file environment and a switchable interface.

• CFF Explorer

- First PE editor with full support for the .NET file format.
- With this tool you can easily edit metadata's fields and flags.
- If you're programming something that has to do with .NET metadata, you will need this tool.
- The resource viewer supports .NET image formats like icons, bitmaps, pngs.
- You'll be able to analyze .NET files without having to install the .NET framework, this tool has its own functions to access the .NET format.

Resource Hacker

- Extracts embedded resources like icons, dialogs, strings.
- Useful for identifying hidden or suspicious content.

Resource
Hacker

Resource Hacker

- Resource Hacker is a resource editor for 32bit and 64bit Windows® applications.
- It's both a
 - ~ resource compiler (for *.rc files)
 - ~ decompiler
 - enabling viewing and editing of resources in
 - ~ executables (*.exe; *.dll; *.scr; etc) and
 - ~ compiled resource libraries (*.res, *.mui).
- While Resource Hacker is primarily a GUI application, it also provides many options for compiling and decompiling resources from the command-line.

Resource
Hacker

Malware Signatures

- A malware signature is a unique pattern such as a hash, string, byte sequence, behavior, or rule that is used to identify malicious software.
- Signatures identify known malware patterns.
- Based on byte sequences or behavior traits.

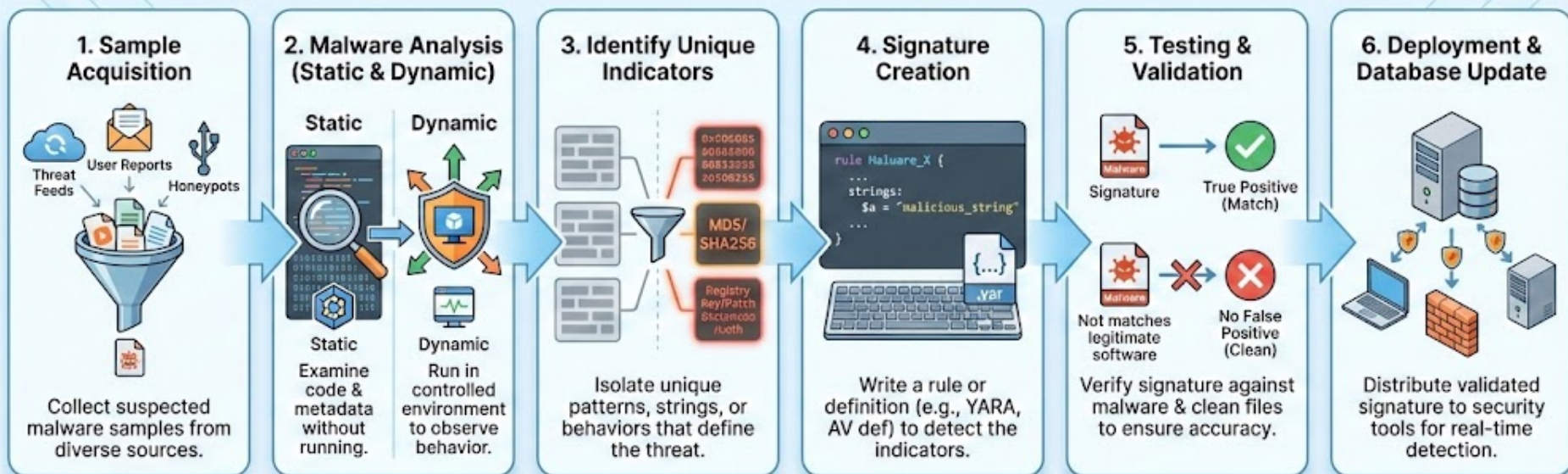
Malware Signatures: Who Creates?

- Antivirus & cybersecurity companies
- Security researchers & malware analysts
- Open-source security communities
- Government CERTs
- Enterprise SOC teams
- Automated & AI-based systems

Malware Signatures: How?

PROCESS OF CREATING A MALWARE SIGNATURE

From Sample Acquisition to Deployment for Threat Detection.



ClamAV



- ClamAV is an open source (GPLv2) anti-virus toolkit, designed especially for e-mail scanning on mail gateways.
- It provides a number of utilities including
 - ~ a flexible and scalable multi-threaded daemon,
 - ~ a command line scanner and
 - ~ advanced tool for automatic database updates.
- The core of the package is an anti-virus engine available in a form of shared library.
 - ~ Detects trojans, viruses, malware & other malicious threats
- Tip: **ClamAV is not a traditional anti-virus or endpoint security suite.**

ClamAV

- ClamAV's bytecode signature runtime, powered by either LLVM or our custom bytecode interpreter, allows the ClamAV signature writers to create and distribute very complex detection routines and remotely enhance the scanner's functionality.
- Signed signature databases ensure that ClamAV will only execute trusted signature definitions.

ClamAV Virus Signatures

- Open-source antivirus signature database.
- Used for detecting known malware samples.
- ClamAV signatures are primarily text-based and conform to one of the ClamAV-specific signature formats associated with a given method of detection.
- The ClamAV project distributes a collection of signatures in the form of CVD (ClamAV Virus Database) files.
- The CVD file format provides a digitally-signed container that encapsulates the signatures and ensures that they can't be modified by a malicious third-party.
- This signature set is actively maintained by Cisco Talos and can be downloaded using the freshclam application that ships with ClamAV.

ClamAV Virus Signatures

Creating ClamAV Signatures

1. Signature Basics

Definition:
Patterns to detect
threats
(Text or YARA)

Distribution:
Official .CVD files
(Signed containers)

Custom Signatures:
Add .hdb, .ldb, .yar files
to database directory

2. Signature Types (File Extensions)

Hash-based

.hdb (MD5/SHA hash)

.mdb
(PE section hash)

Body-based

.ndb (Extended)

.ldb (Logical)

.cbc (Bytecode)

Other

.yar (YARA rules)

.fp (False positives)

.ign
(Ignored signatures)

3. Creation & Testing Workflow

STEP 1: Write Signature
Box: Create text file with rule, use correct
format/extension

STEP 2: Test Signature
Box: Run 'clamscan -d my_sig.ldb sample_file'

STEP 3: Debug & Optimize

- Use 'clamscan -debug -leave-temps' to view unpacking/normalization
- Check temp files for matching against normalized content

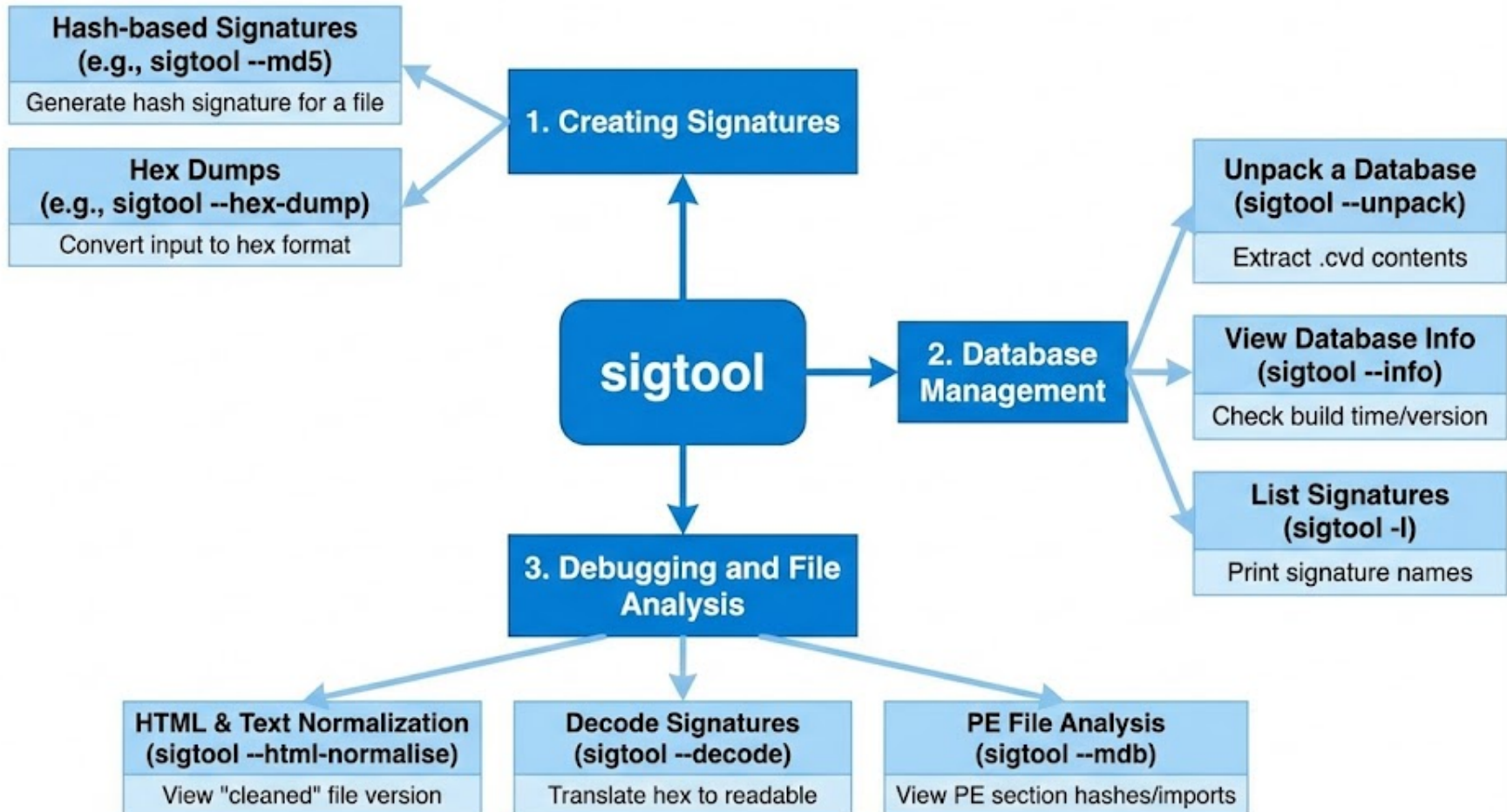
Tools: sigtool (-md5, -html-normalise, -unpack)

Naming Convention

{platform}.{category}.{name}-{signature id}-{revision}
Alphanumeric, dash, dot, underscore only

ClamAV Virus Signatures

Sigtool Utility Overview



• YARA

- YARA (Yet Another Recursive Acronym) is a tool primarily used by malware researchers to identify and classify malware samples.
- It works by creating descriptions (rules) that look for specific characteristics in a file—typically strings (text or hex) and boolean conditions.
- YARA rules are a pattern-matching language used to identify and classify malware (and other file characteristics) by describing content, structure, and behavior.
- They are widely used in malware research, incident response, and are supported by ClamAV as an additional signature mechanism.

- YARA

- Detect malware families or variants based on byte patterns, strings, and logical conditions.
- Complement traditional hash-based and byte-signature detection, especially for polymorphic or obfuscated threats.

• YARA Rule Structure

- A YARA rule is composed of three main sections:
- Meta – Descriptive information (author, date, reference, version).
- Strings – Text, hex patterns, or regular expressions to search for.
- Condition – Logical expression defining when the rule matches (e.g., combinations, counts, offsets, file size checks).

```
rule Detect_Suspicious_Downloader
```

```
{
```

```
/* SECTION 1: META
```

```
Metadata is purely descriptive. It helps analysts organize rules  
but does not affect the scanning logic itself.
```

```
*/
```

```
meta:
```

```
author = "CyberOps Team"
```

```
description = "Detects executables that attempt to download payloads"
```

```
date = "2026-01-19"
```

```
severity = "High"
```

```
hash_reference = "d41d8cd98f00b204e9800998ecf8427e"
```

```
/* SECTION 2: STRINGS
```

```
Defines the patterns to search for in the file.
```

```
These can be text (ASCII/Wide), Hexadecimal, or Regex.
```

```
*/
```

```
strings:
```

```
// ASCII string that might appear in the code
```

```
$cmd_url = "http://malicious-site.com/payload.exe" ascii wide nocase
```

```
// A specific function call often seen in this malware family
```

```
$func_call = "DownloadToFileA"
```

```
// A Hex pattern representing a specific machine code sequence (wildcards included)
```

```
// E.g., specific instructions at the entry point
```

```
$hex_signature = { E8 ?? ?? ?? ?? 8B 45 08 50 }
```

```
/* SECTION 3: CONDITION
```

```
The boolean logic that determines if the rule matches.
```

```
All criteria here must evaluate to TRUE for a detection to occur.
```

```
*/
```

```
condition:
```

```
// Must be a Windows PE file (starts with "MZ" magic bytes)
```

```
uint16(0) == 0x5A4D and
```

```
// The file must be small (less than 1MB), typical for droppers
```

```
filesize < 1000KB and
```

```
// Logic: Must match the hex signature AND at least one of the text strings
```

```
$hex_signature and ($cmd_url or $func_call)
```

```
}
```


YARA Signatures

- Rule-based malware identification language.
- Used by analysts to detect malware families.

Conclusion

- Static analysis is a foundational malware analysis technique.
- Combines tools, signatures, and file inspection.